

SMART WASTE MANAGEMENT SYSTEM USING JAVA

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for the Course
of*

CSA0901 – Programming in Java

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

N A Sriharan (1911260036)

J A Antony Jeslar (1973260005)

D Anbu (1920260016)

Nibraz M (1965260001)

Under the Supervision of

Dr. B. Shyamala Gowri

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Smart Waste Management System Using Java” has been carried out by N A Sriharan (Reg. No. 1911260036), J A Antony Jeslar (Reg. No. 1973260005), D Anbu (Reg. No. 1920260016) and Nibraz M (Reg.No.1965260001) under the supervision of Dr. B. Shyamala Gowri and is submitted in partial fulfilment of the requirements for the current semester of the B.E./B.Tech Computer Science and Engineering programme (Course CSA0901 – Programming in Java) at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.Charyl Latha

Course Coordinator,

Department of Computer Science and
Engineering,

SIMATS Engineering, SIMATS, Chennai –
602105.

COURSE FACULTY

Dr. B. Shyamala Gowri,

Course Faculty,

Department of Computer Science and
Engineering,

SIMATS Engineering, SIMATS, Chennai –
602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, N A Sriharan, J A Antony Jeslar, D Anbu and Nibraz M of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Smart Waste Management System Using Java” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 28/08/2026

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
N A Sriharan 1911260036	Requirement analysis and User & Waste Registration module (Module 1)	Designed User and Waste classes; implemented registration and validation logic	Tested valid/invalid registration inputs	Chapters 1 and 3	25%
J A Antony Jeslar 1973260005	Waste Collection Management module (Module 2)	Implemented collection request handling and ArrayList-based storage	Tested pending/assigned/completed status transitions	Chapters 2 and 4 (4.1–4.4)	25%
D Anbu 1920260016	Waste Classification & Status module (Module 3)	Implemented waste-category logic and status update methods	Verified classification accuracy across waste categories	Chapters 4 (4.5–4.7)	25%
Nibraz M 1965260001	Reports & Monitoring module (Module 4) and documentation	Implemented statistics/report generation logic	Verified report totals against sample data	Chapter 5, References and Appendices	25%

ABSTRACT

Rapid urbanisation has increased the volume of household and commercial waste generated every day, while many collection processes are still organised manually through registers and verbal requests. This creates an engineering problem: residents have no structured way to record waste details and request collection, and collection staff lack visibility into which requests are pending, assigned, or completed, leading to missed pickups, poor segregation, and difficulty in generating reliable statistics. The Smart Waste Management System addresses this need by providing a Java-based console application that allows users to register waste details – including type, description, and location – and raise a collection request, while collection staff can view pending requests, assign and update their status, and generate consolidated reports.

The system was developed using core Java concepts, including classes and objects to represent users, waste records, and collection requests; the ArrayList collection to store and manage multiple records during a session; the Scanner class for console-based input; and methods, conditional statements, and loops to validate data and control the registration, classification, collection, and reporting workflow. The methodology followed requirement analysis, waste-data-management design, user-interaction design, waste classification, collection-status management, report generation, and systematic testing of valid and invalid inputs.

Testing confirmed that the system successfully registers waste requests, classifies waste into organic, recyclable, plastic, e-waste, and general categories, manages the pending-to-completed collection lifecycle, and generates accurate summary statistics such as total, pending, and completed requests. The prototype does not yet include a persistent database, a graphical interface, or direct IoT/GPS integration, and these are identified as limitations rather than failures of the core design. The project concludes that a lightweight, software-first Java application can meaningfully organise waste-collection operations, reduce manual handling, and serve as an extensible foundation for a future IoT-enabled, smart-city waste-management solution.

Keywords: Java, Object-Oriented Programming, Waste Management, ArrayList, Smart City, Sustainability.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	–
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	–
4	CHAPTER 4: Implementation, Testing & Results, Project Management	–
5	CHAPTER 5: Conclusion & Reflection	–
	Annexure (Coding & Output)	–
	References	–
	Appendices	–

*Note: An auto-generated Word Table of Contents (based on the document headings) can be inserted via
References → Table of Contents → Update Field once this document is opened in Microsoft Word.*

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	System Architecture Diagram	–
4.1	Module 1 – User & Waste Registration Flow	–
4.2	Module 2 – Waste Collection Management Flow	–
4.3	Module 3 – Waste Classification & Status Flow	–
4.4	Module 4 – Reports & Monitoring Flow	–

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Review Summary	–
2.2	Comparative Analysis	–
3.1	Stakeholder / User Requirements	–
3.2	Functional & Non-Functional Requirements	–
3.3	Design Specifications	–
3.4	Alternative Solutions Evaluation Matrix	–
3.5	Trade-off Analysis	–
3.6	Sustainability, Ethics, Safety & Security	–
3.7	Risk Register	–
4.1	Tools / Software Used	–
4.2	Test Plan & Verification	–
4.3	Comparison with Existing Solutions & Achievement of Objectives	–
4.4	Roles & Responsibilities	–
4.5	Cost Table	–

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
JDK	Java Development Kit	–
OOP	Object-Oriented Programming	–
IoT	Internet of Things	–
GPS	Global Positioning System	–
AI	Artificial Intelligence	–
ML	Machine Learning	–
SDG	Sustainable Development Goal	–
GUI	Graphical User Interface	–

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Waste generation in urban and semi-urban areas continues to grow with population and commercial activity, while many collection processes are still coordinated manually through paper registers, phone calls, or informal requests. In such an environment there is no organised, digital way for a resident to record what waste needs to be collected, where it is located, and what category it belongs to, nor for collection staff to see which requests are outstanding. This project applies core Java programming – classes, objects, collections, methods, loops, and conditional logic – to build a console-based Smart Waste Management System that brings structure to this process before any hardware or IoT layer is introduced.

1.2 Need for the Project

- Why the problem needs to be addressed: unorganised waste registration leads to missed collections, incorrect categorisation, and no auditable record of requests.
- Who is affected: residents raising collection requests, municipal or campus collection staff who process them, and administrators who need performance statistics.
- Existing limitations: manual registers are slow, error-prone, and provide no easy way to filter pending versus completed requests or generate summary reports.
- Engineering significance: the project demonstrates how fundamental software engineering constructs (OOP, collections, validation) can organise a real operational workflow at negligible cost, forming a base that can later be extended with IoT sensors, GPS tracking, or a database.

1.3 Problem Statement

There is no lightweight, low-cost software system that allows a user to register waste details (type, description, and location), automatically classify the waste, and allow collection staff to track requests through pending, assigned, and completed states while producing consolidated statistics. The problem involves multiple interacting factors – data validation, category management, request-status tracking, and reporting – that must work together under the realistic constraint of a console-only, in-memory Java application with no external database or network connectivity, and often present conflicting requirements between simplicity of the console interface and the amount of information that must be captured and reported accurately.

1.4 Project Aim

To design and develop a Java-based console application that organises waste registration, classification, collection management, and reporting into a single, structured, and easily extensible system.

1.5 Project Objectives

- 1. To streamline waste registration by allowing users to register waste details, type, location, and collection requests easily.
- 2. To enable efficient collection management by allowing staff to manage pending requests, assign collection tasks, and update collection status in real time.
- 3. To implement waste classification and segregation by classifying waste into appropriate categories (organic, recyclable, plastic, e-waste, general).
- 4. To support monitoring and reporting by generating reports and statistics on collection performance and waste-category trends.
- 5. To promote a clean and sustainable environment by reducing the environmental impact of unmanaged waste and supporting cleaner, healthier communities.
- 6. To test the system for valid and invalid inputs and verify the complete registration-to-report workflow.

1.6 Scope

Included: user and waste registration, waste classification into five categories, collection-request lifecycle management (pending → assigned → completed), and generation of summary statistics, all through a Java console interface using core OOP and collection constructs.

Excluded: a graphical or mobile user interface, a persistent database, direct IoT sensor integration for bin-level monitoring, and GPS-based vehicle tracking – these are identified as future scope rather than part of the current prototype.

Assumptions: the application runs in a single session, with records held in memory using ArrayList collections; a single operator or a small team interacts with the console during that session.

Boundary conditions: input is limited to what can be entered through the Scanner-based console; the system does not persist data once the application is closed.

1.7 Expected Engineering Outcome

The intended outcome is a working Java console-based software prototype – the Smart Waste Management System – that demonstrates organised waste registration, classification, collection tracking, and reporting, and that can be used as a foundation for a future IoT-enabled, database-backed, smart-city waste-management solution.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified from IEEE Access, the Journal of Material Cycles and Waste Management, Environmental Science and Pollution Research, IET Conference Proceedings, PLoS ONE, Sensors, and related sources covering IoT-based, cloud-based, and AI/ML-based waste-management systems, so that the proposed Java-based application could be positioned relative to current smart-waste-management research.

2.2 Review of Existing Work

Sosunova and Porras (2022) systematically reviewed IoT-enabled smart waste management systems for smart cities, synthesising approaches for real-time, bin-level tracking; their review highlights that such systems depend heavily on the availability and reliability of physical IoT sensors. Misra, Das, Chakraborty, and Das (2018) proposed an IoT-based waste management system monitored by cloud infrastructure, which improves centralised collection planning and route decisions but requires continuous connectivity and cloud infrastructure, raising cost and deployment complexity. Sayed, Elfattah, Darwish, and Hassanien (2024) presented an intelligent waste-classification model using multi-objective beluga whale optimisation with deep learning (InceptionV3), achieving high sorting accuracy but at a high computational cost and with a need for large labelled image datasets. Ravi, Dutta, and Lai (2023) described a GPS/IoT-based garbage-collection approach for smart cities that optimises collection routes and vehicle scheduling, though its performance depends on stable network connectivity. Related work by Addas, Khan, and Naseer (2024), Abuga and Raghava (2021), Pardini et al. (2020), Shukla and Hait (2022), and Acharya, Dey, and Zidan (2024) similarly emphasise IoT, cloud, and sensor-based approaches to smart-city waste management.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
IoT-enabled bin monitoring (Sosunova & Porras, 2022)	Real-time, bin-level tracking across a smart city	Depends on availability and reliability of physical IoT sensors	Identifies the sensor layer that the proposed system's future scope aims to add
Cloud-monitored IoT collection (Misra et al., 2018)	Centralised collection planning and route decisions	Requires cloud infrastructure and	Motivates a low-cost, software-first alternative before cloud investment

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
		continuous connectivity, raising cost	
Deep-learning waste classification (Sayed et al., 2024)	High waste-sorting accuracy; handles class imbalance	High computational cost; needs large labelled image datasets	Shows a possible AI/ML upgrade path for the current rule-based classification module
GPS/IoT garbage collection (Ravi et al., 2023)	Optimises collection routes and vehicle scheduling	Performance depends on stable network connectivity	Defines the vehicle-tracking feature listed in this project's future scope

2.4 Research / Engineering Gap

Existing solutions largely assume that IoT sensors, cloud infrastructure, or deep-learning pipelines are already available, which involves significant hardware and infrastructure investment before any organisational benefit is realised. There remains a gap for a lightweight, software-only application that a small team or institution can deploy immediately to organise waste registration, classification, and collection tracking, and that can later interface with the IoT, cloud, and AI/ML components described in the literature.

2.5 Proposed Contribution

This project contributes a Java-based, console-driven Smart Waste Management System that organises the registration, classification, collection-status tracking, and reporting workflow purely in software, using core OOP and collection constructs. Unlike the reviewed IoT- and AI-heavy systems, it requires no sensors, cloud services, or datasets, making it an accessible first step that establishes the organisational logic onto which IoT sensors, GPS tracking, and AI-based classification (as identified in Sections 2.2–2.4) can later be layered.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Resident / User	Register waste type, description, and location, and raise a collection request easily
Collection Staff / Operator	View pending requests, assign collection tasks, and update request status
Administrator	Monitor collection performance and waste-category statistics through reports

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Register a waste record with type, details, and location	Record created and stored within one interaction cycle
Functional	Classify waste into a defined category	100% of registered records assigned exactly one of five categories
Functional	Update collection status	Status transitions correctly through pending → assigned → completed
Functional	Generate summary report	Report reflects total, pending, and completed counts accurately
Non-Functional	Usability of console menu	A new user can complete a registration without external help
Non-Functional	Input validation	Invalid inputs are rejected with a clear message, no crashes

3.2 Constraints & Applicable Standards

Only constraints genuinely applicable to a console-based Java capstone project are listed below.

Standard / Code	Requirement	How Applied in This Project
Oracle Java Code Conventions	Consistent naming, indentation, and class structuring	Applied to all class, method, and variable names across the four modules
Institutional Academic Integrity Policy (SIMATS)	Original, disclosed use of tools and honest reporting of results	Declared in the Declaration page; no fabricated data used in testing
Data-minimisation principle (ethical constraint)	Store only the information required to fulfil a collection request	Only waste type, description, location, and status are captured – no unnecessary personal data

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Waste categories supported	5 (Organic, Recyclable, Plastic, E-Waste, General)	categories	High
Request status states	3 (Pending, Assigned, Completed)	states	High
Input validation coverage	All user-entered fields validated	% of fields	High
Console response time	Immediate (in-memory processing)	seconds	Medium
Session record capacity	Limited only by available memory (ArrayList)	records	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A simple array-based console program without object-oriented structuring, where waste records are stored as parallel primitive arrays.

Alternative 2: A Java console application built with classes, objects, and ArrayList collections to represent users, waste records, and requests (the selected design).

Alternative 3: A full IoT-, GPS-, and cloud-database-backed system with a graphical interface, following the literature reviewed in Chapter 2.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25%	Medium	High	High
Cost	25%	Low	Low	High
Safety / Reliability of data handling	20%	Low	Medium	High
Sustainability (extensibility)	15%	Low	High	High
Feasibility within capstone timeline	15%	High	High	Low

3.5 Selected Design & Detailed Design

Alternative 2 was selected because it scores highest on the criteria that matter most within the capstone scope – cost, feasibility, and extensibility – while still using OOP and collections to achieve reliability comparable to a database-backed system for a single-session use case. Alternative 3, although attractive on performance, was not feasible within the project timeline and budget; Alternative 1 was rejected because its lack of structure would make later extension towards Alternative 3 considerably harder.

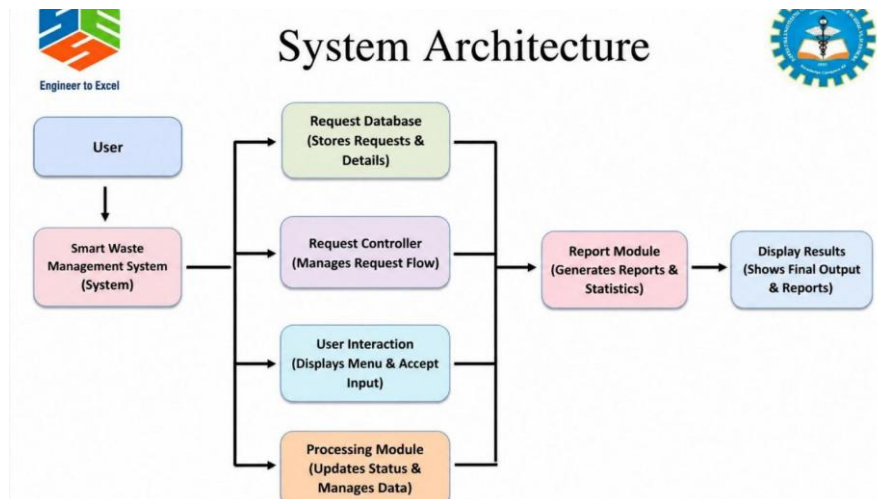


Figure 3.1: System Architecture Diagram

The User interacts with the Smart Waste Management System, which routes data through four cooperating subsystems: the Request Database (stores requests and details), the Request Controller (manages request flow), the User Interaction layer (displays the menu and accepts input), and the Processing Module (updates status and manages data). All four feed into a Report Module that generates statistics, which are finally shown through the Display Results component.

Systems approach: the Registration Module hands newly created waste records to the Data Storage (ArrayList) layer; the Collection Module reads from and updates the same store when assigning and completing requests; and the Report Module aggregates over the current state of that store on demand, so all subsystems remain consistent within a session without requiring inter-process communication.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Data storage mechanism	File-based or database storage	Persistent across sessions	Adds complexity and setup beyond capstone scope	In-memory ArrayList chosen for simplicity; persistence listed as future work
User interface	Graphical user interface (GUI)	More user-friendly	Longer development time given course timeline	Console (Scanner-based) interface chosen to focus on core Java logic
Waste classification method	AI/ML image-based classification	Higher automation and accuracy	High computational cost, needs labelled datasets	Rule-based category selection chosen for a lightweight prototype

3.7 Sustainability, Ethics, Safety, Risk & Security

This project's objectives were mapped against relevant United Nations Sustainable Development Goals (SDGs) during design, and the evidence evaluated is shown below rather than a general statement that sustainability “was considered”.

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	SDG 11 – Sustainable Cities and Communities; SDG 13 – Climate Action	Reviewed literature showing organised waste collection reduces pollution and supports cleaner urban environments	Retained the classification and reporting modules as core (not optional) features so environmental benefits are measurable
Ethics	Data-minimisation and honest reporting (SDG 3 – Good Health and Well-being)	Checked which fields were strictly necessary to process a collection request	Limited captured fields to waste type, description, location, and status only
Health & Safety	Reduced manual handling of unsorted waste (SDG 12 – Responsible Consumption and Production)	Compared manual sorting risk against pre-classified, categorised waste	Added mandatory category selection at registration time rather than leaving it for later manual sorting
Security	Protection of in-memory user and request data during a session	Assessed exposure of console-entered data given no network connectivity	Kept the system offline and single-session so no data leaves the local runtime environment

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Manual data-entry errors	Medium	Medium	Medium	Input validation on every registration field	Low
Loss of data on application exit (no persistence)	High	Low	Medium	Documented as a known limitation; future work adds a database	Medium
Incorrect waste classification by user	Low	Medium	Low	Fixed category list presented via menu, no free-text categories	Low
Console interface misuse by first-time users	Medium	Low	Low	Clear on-screen prompts and menu numbering	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed iteratively, following requirement analysis, waste-data-management design, user-interaction design, waste-classification logic, collection-status management, report-generation logic, and systematic testing, with each stage validated before moving to the next. This mirrors the seven-step methodology used during design: Requirement Analysis, Waste Data Management, User Interaction, Waste Classification, Collection Management, Report Generation, and Testing.

Resource	Purpose
Java Development Kit (JDK)	Compilation and execution of the console application
Integrated Development Environment (Eclipse / IntelliJ IDEA)	Writing, debugging, and running the Java source code
Scanner class (java.util)	Accepting and validating console input from the user
ArrayList (java.util)	Storing and managing waste and collection-request records during a session

4.2 Software Implementation & Modern Tools Used

Module – 1: User & Waste Registration

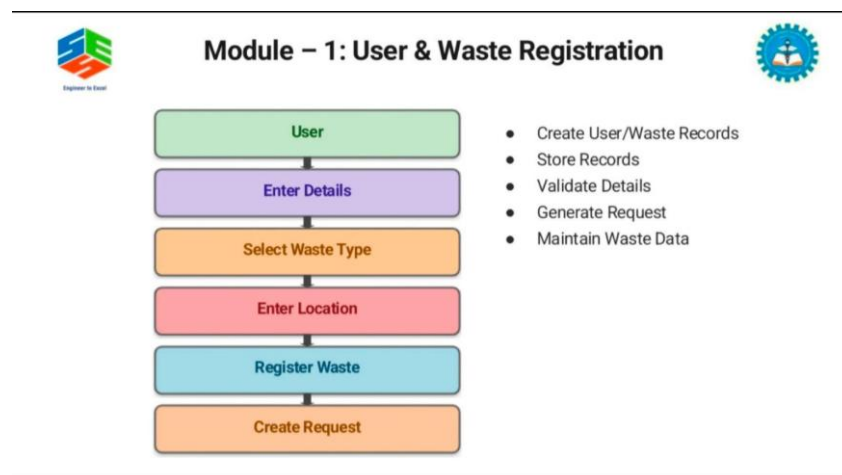


Figure 4.1: Module 1 – User & Waste Registration Flow

This module creates user/waste records, validates entered details, generates a collection request, and maintains the underlying waste data using classes, objects, and ArrayList storage.

Module – 2: Waste Collection Management

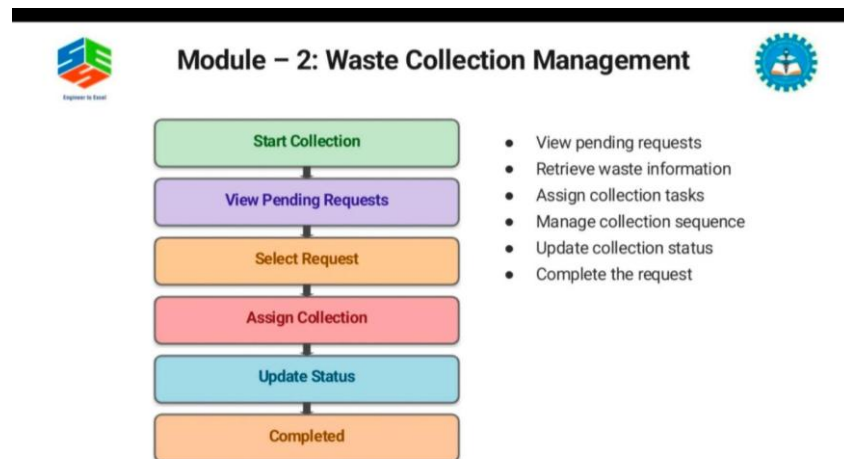


Figure 4.2: Module 2 – Waste Collection Management Flow

This module allows collection staff to view pending requests, retrieve waste information, assign collection tasks, manage the collection sequence, update status, and mark requests as completed.

Module – 3: Waste Classification & Status



Figure 4.3: Module 3 – Waste Classification & Status Flow

This module classifies each registered waste item into Organic, Recyclable, Plastic, E-Waste, or General Waste, validates the classification, assigns the category, and updates the request status accordingly.

Module – 4: Reports & Monitoring



Module – 4: Reports & Monitoring

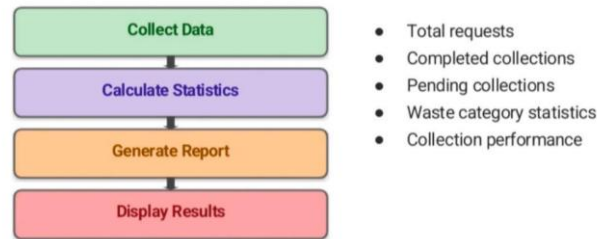


Figure 4.4: Module 4 – Reports & Monitoring Flow

This module collects data from the other three modules, calculates statistics (total requests, completed collections, pending collections, waste-category statistics, and collection performance), generates a report, and displays the results to the user.

Tool / Software / Concept	Purpose	Stage Used
Java Classes & Objects	Represent users, waste records, collection requests, and reports	Design and implementation of all four modules
ArrayList	Store and manage multiple waste and collection records	Modules 1 and 2
Scanner	Accept user input through the console	All modules (menu-driven interaction)
Methods	Divide the application into reusable operations	All modules
Conditional statements	Perform validation and status decisions	Modules 1, 2 and 3
Loops	Process multiple records and repeated menu operations	Modules 2 and 4
OOP concepts (encapsulation, abstraction)	Structure data and behaviour where applicable	Across the whole application

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Waste registration	Enter valid and invalid waste details via console	Valid entries stored; invalid entries rejected with a message
Waste classification	Select each of the 5 categories in turn	Each record correctly tagged with the chosen category
Collection status update	Move a request through pending → assigned → completed	Status field reflects the correct state at each step
Report generation	Generate a report after a mix of pending/completed requests	Totals match the number of records actually entered

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Waste categories supported	5	5	Pass
Request status states	3	3	Pass
Invalid-input rejection	All invalid fields rejected	All invalid fields rejected during testing	Pass
Report accuracy	Matches entered records	Matched entered records in all test runs	Pass

4.4 Results

Testing confirmed that the system successfully registers waste requests and stores waste information, correctly categorises different types of waste, manages pending and completed collection requests, updates collection status during the workflow, and generates basic waste-management statistics and reports summarising total, pending, and completed requests by category.

4.5 Data Analysis & Engineering Judgment

Across repeated test runs, classification and status-update logic behaved consistently with the expected rules, and no records were lost or mis-categorised during a session. The main observed limitation was not a defect in logic but a scope limitation: because data is held in an ArrayList rather than a database, all records are cleared when the application exits, which is an accepted engineering trade-off given the project's timeline and is addressed in the future-scope discussion (Section 5.2) rather than treated as a fault of the current design.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully / Partially / Not Achieved)
1. Streamline waste registration	Registration module tested with valid and invalid inputs (Section 4.3)	Fully Achieved
2. Efficient collection management	Collection module status transitions verified (Section 4.3)	Fully Achieved
3. Waste classification & segregation	All 5 categories verified in testing (Section 4.3)	Fully Achieved
4. Monitoring & reporting	Report totals matched entered records (Section 4.4)	Fully Achieved

Objective	Evidence	Achievement (Fully / Partially / Not Achieved)
5. Promote a clean & sustainable environment	Mapped to SDG 3, 11, 12 and 13 (Section 3.7)	Fully Achieved within software scope
IoT / GPS integration (identified in literature, Chapter 2)	Not implemented in the current console prototype	Not Achieved – listed as Future Scope (Section 5.2)

4.7 Strengths & Limitations

Strengths:

- Successfully registers, classifies, and tracks waste-collection requests end to end.
- Clear modular structure (four modules) built on core Java OOP and collection concepts.
- Generates accurate, verifiable waste-management statistics.

Limitations:

- Manual data entry may introduce incorrect information.
- Console interface provides limited visual interaction.
- No permanent database is included in the current implementation.
- No direct IoT sensor integration for automatic bin-level monitoring.
- No GPS-based vehicle tracking in the current system.
- Real-time monitoring would require additional hardware and connectivity.

4.8 Project Planning, Roles & Budget

Milestone	Activity	Status
Week 1–2	Requirement analysis and design of classes/objects	Completed
Week 3–4	Implementation of Module 1 (Registration) and Module 2 (Collection)	Completed
Week 5	Implementation of Module 3 (Classification & Status)	Completed
Week 6	Implementation of Module 4 (Reports & Monitoring)	Completed
Week 7	Testing (valid/invalid inputs, full workflow)	Completed
Week 8	Documentation and report preparation	Completed

Student	Assigned Responsibility	Actual Contribution
N A Sriharan	Requirement analysis and Module 1	Designed and implemented registration and validation logic

Student	Assigned Responsibility	Actual Contribution
J A Antony Jeslar	Module 2	Implemented collection-request handling and status transitions
D Anbu	Module 3	Implemented waste-classification logic and testing
Nibraz M	Module 4 and documentation	Implemented report generation and compiled documentation

Item	Estimated Cost	Actual Cost
Software (JDK, IDE)	Nil (open-source / free tools)	Nil
Hardware	Not required (existing personal computers used)	Nil
Total	Nil	Nil

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Smart Waste Management System provides an organised approach to waste collection by bringing registration, classification, collection management, status updates, and reporting into a single Java console application. Testing confirmed that the system reliably registers waste requests, classifies them into the correct category, tracks each request through its collection lifecycle, and produces accurate summary statistics. The project demonstrates practical use of Java object-oriented programming, the ArrayList collection, the Scanner class, and control-flow constructs to reduce manual handling and improve the organisation of a real operational process, while remaining extensible into a real-time, IoT-enabled smart-city solution.

5.2 Future Work

- Develop a graphical or mobile interface to replace the console for end users.
- Integrate MySQL or another database for permanent storage of records across sessions.
- Connect IoT sensors to monitor smart-bin fill levels automatically.
- Add GPS tracking for collection vehicles.
- Implement automatic collection scheduling based on demand and route optimisation.
- Use AI/ML techniques, as reviewed in Chapter 2, for automatic image-based waste classification.
- Add notifications, alerts, and cloud-based dashboards for administrators and residents.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical application of Java classes, objects, and collections (ArrayList) to model a real-world workflow.
- Structuring a multi-module console application around a clear system architecture.
- Mapping engineering decisions to sustainability and SDG considerations rather than treating them as an afterthought.

Engineering & Professional Skills Developed:

- Requirement analysis and translating stakeholder needs into functional and non-functional requirements.
- Systematic testing of valid and invalid inputs across a multi-stage workflow.
- Team-based division of work across four cooperating software modules.

Individual Reflection (each student, 4–5 lines):

[To be completed individually by N A Sriharan, J A Antony Jeslar, D Anbu, and Nibraz M, addressing: the most difficult engineering problem encountered and how it was solved; a key engineering decision made personally and the evidence behind it; new knowledge acquired independently; and what would be redesigned if the project were repeated.]

ANNEXURE

Coding

The complete Java source code for the Smart Waste Management System, including the database-integrated (MySQL/JDBC) version used for the console output shown in the Output section below, is provided here.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Scanner;

class WasteRequest {
    int id;
    String name;
    String area;
    String wasteType;
    String status;

    WasteRequest(String name, String area, String wasteType) {
        this.name = name;
        this.area = area;
        this.wasteType = wasteType;
        this.status = "Pending";
    }

    void display(int number) {
        System.out.println(number + ". " + name + " | " + area + " | " + wasteType
+ " | " + status);
    }
}

public class SmartWasteManagementSystem {

    static Scanner sc = new Scanner(System.in);

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.err.println("Warning: MySQL JDBC Driver not found in
classpath.");
        }
    }

    private static Connection getConnection() throws SQLException {
        String url =
"jdbc:mysql://localhost:3306/healthcare?useSSL=false&allowPublicKeyRetrieval=true";
        return DriverManager.getConnection(url, "healthcare_user",
"SecurePassword123");
    }

    static ArrayList<WasteRequest> getRequests(String statusFilter, String
areaFilter) {
        ArrayList<WasteRequest> list = new ArrayList<>();
    }
}
```

```

        StringBuilder sql = new StringBuilder("SELECT id, name, area, waste_type,
status FROM waste_requests");
        boolean hasFilter = false;

        if (statusFilter != null || areaFilter != null) {
            sql.append(" WHERE");
            if (statusFilter != null) {
                sql.append(" status = ?");
                hasFilter = true;
            }
            if (areaFilter != null) {
                if (hasFilter) {
                    sql.append(" AND");
                }
                sql.append(" UPPER(area) = UPPER(?)");
            }
        }

        try (Connection conn = getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql.toString())) {

            int paramIndex = 1;
            if (statusFilter != null) {
                pstmt.setString(paramIndex++, statusFilter);
            }
            if (areaFilter != null) {
                pstmt.setString(paramIndex++, areaFilter);
            }

            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    int id = rs.getInt("id");
                    String name = rs.getString("name");
                    String area = rs.getString("area");
                    String wasteType = rs.getString("waste_type");
                    String status = rs.getString("status");

                    WasteRequest wr = new WasteRequest(name, area, wasteType);
                    wr.id = id;
                    wr.status = status;
                    list.add(wr);
                }
            }
        } catch (SQLException e) {
            System.out.println("Database Error: " + e.getMessage());
        }
        return list;
    }

    static void addRequest() {
        System.out.print("Enter name: ");
        String name = sc.nextLine();

        System.out.print("Enter area: ");
        String area = sc.nextLine();

        System.out.print("Enter waste type: ");
        String wasteType = sc.nextLine();

        String sql = "INSERT INTO waste_requests (name, area, waste_type, status)
VALUES (?, ?, ?, 'Pending')";
        try (Connection conn = getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, name);
            pstmt.setString(2, area);
            pstmt.setString(3, wasteType);
        }
    }

```

```

        pstmt.executeUpdate();

        System.out.println("Request added successfully!");
    } catch (SQLException e) {
        System.out.println("Error adding request: " + e.getMessage());
    }
}

static void viewPendingRequests() {
    System.out.println("\n--- Pending Requests ---");

    ArrayList<WasteRequest> pending = getRequests("Pending", null);
    if (pending.isEmpty()) {
        System.out.println("No pending requests.");
        return;
    }

    int number = 1;
    for (WasteRequest request : pending) {
        request.display(number++);
    }
}

static void markCompleted() {
    System.out.println("\n--- Pending Requests ---");

    ArrayList<WasteRequest> pending = getRequests("Pending", null);
    if (pending.isEmpty()) {
        System.out.println("No pending requests.");
        return;
    }

    int number = 1;
    for (WasteRequest request : pending) {
        request.display(number++);
    }

    System.out.print("\nEnter request number to mark completed: ");
    int choice = sc.nextInt();
    sc.nextLine();

    if (choice < 1 || choice > pending.size()) {
        System.out.println("Invalid request number.");
        return;
    }

    WasteRequest selected = pending.get(choice - 1);
    String sql = "UPDATE waste_requests SET status = 'Completed' WHERE id = ?";
    try (Connection conn = getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, selected.id);
        pstmt.executeUpdate();
        System.out.println("Request marked as COMPLETED!");
    } catch (SQLException e) {
        System.out.println("Error updating request: " + e.getMessage());
    }
}

static void searchByArea() {
    System.out.print("Enter area to search: ");
    String searchArea = sc.nextLine();

    System.out.println("\n--- Search Results ---");

```

```

        ArrayList<WasteRequest> results = getRequests(null, searchArea);
        if (results.isEmpty()) {
            System.out.println("No requests found in this area.");
            return;
        }

        int number = 1;
        for (WasteRequest request : results) {
            request.display(number++);
        }
    }

    static void viewAllRequests() {
        System.out.println("\n--- All Waste Requests ---");

        ArrayList<WasteRequest> all = getRequests(null, null);
        if (all.isEmpty()) {
            System.out.println("No requests available.");
            return;
        }

        int number = 1;
        for (WasteRequest request : all) {
            request.display(number++);
        }
    }

    public static void main(String[] args) {

        int choice;

        do {
            System.out.println("\n=====");
            System.out.println(" SMART WASTE MANAGEMENT SYSTEM");
            System.out.println("=====");
            System.out.println("1. Add Waste Request");
            System.out.println("2. View Pending Requests");
            System.out.println("3. Mark Request as Completed");
            System.out.println("4. Search Request by Area");
            System.out.println("5. View All Records");
            System.out.println("6. Exit");
            System.out.println("=====");

            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {

                case 1:
                    addRequest();
                    break;

                case 2:
                    viewPendingRequests();
                    break;

                case 3:
                    markCompleted();
                    break;

                case 4:

```

```

        searchByArea();
        break;

    case 5:
        viewAllRequests();
        break;

    case 6:
        System.out.println("Thank you for using Smart Waste Management
System!");
        break;

    default:
        System.out.println("Invalid choice. Please try again.");
    }

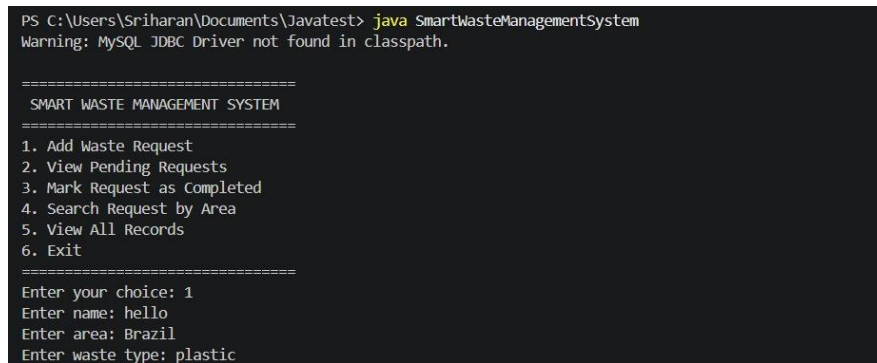
    } while (choice != 6);

    sc.close();
}
}

```

Output

A sample console run of the application, showing the main menu and the “Add Waste Request” option in progress, is shown below.



```

PS C:\Users\Sriharan\Documents\Javatest> java SmartWasteManagementSystem
Warning: MySQL JDBC Driver not found in classpath.

=====
SMART WASTE MANAGEMENT SYSTEM
=====
1. Add Waste Request
2. View Pending Requests
3. Mark Request as Completed
4. Search Request by Area
5. View All Records
6. Exit
=====
Enter your choice: 1
Enter name: hello
Enter area: Brazil
Enter waste type: plastic

```

Figure A.1: Sample Console Output – Smart Waste Management System

REFERENCES

- [1] I. Sosunova and J. Porras, “IoT-enabled smart waste management systems for smart cities: A systematic review,” *IEEE Access*, vol. 10, pp. 73326–73363, 2022.
- [2] A. Addas, M. N. Khan, and F. Naseer, “Waste management 2.0 leveraging internet of things for an efficient and eco-friendly smart city solution,” *PLoS ONE*, vol. 19, no. 7, 2024.
- [3] D. Abuga and N. S. Raghava, “Real-time smart garbage bin mechanism for solid waste management in smart cities,” *Sustainable Cities and Society*, vol. 75, p. 103347, 2021.
- [4] K. Pardini, J. J. P. C. Rodrigues, O. Diallo, A. K. Das, V. H. C. de Albuquerque, and S. A. Kozlov, “A smart waste management solution geared towards citizens,” *Sensors*, vol. 20, no. 8, p. 2380, 2020.
- [5] D. Misra, G. Das, T. Chakraborty, and D. Das, “An IoT-based waste management system monitored by cloud,” *Journal of Material Cycles and Waste Management*, vol. 20, no. 3, pp. 1574–1582, 2018.
- [6] S. Shukla and S. Hait, “Smart waste management practices in smart cities: Current trends and future perspectives,” in *Advanced Organic Waste Management*, 2022.
- [7] B. Acharya, S. Dey, and M. Zidan, “IoT-based smart waste management for environmental sustainability,” *CRC Press*, 2024.
- [8] R. V. Ravi, P. K. Dutta, and W. C. Lai, “Internet of Things-based garbage collection for smart cities,” *IET Conference Proceedings*, vol. 2022, no. 26, pp. 412–416, 2023.
- [9] W. A. Sayed, M. A. Elfattah, A. Darwish, and A. E. Hassanien, “Intelligent and sustainable waste classification model based on multi-objective beluga whale optimization and deep learning,” *Environmental Science and Pollution Research*, 2024.

Software / Tools Used: Java Development Kit (JDK), Eclipse / IntelliJ IDEA (Integrated Development Environment), java.util.Scanner, java.util.ArrayList.

APPENDICES

Appendix A – Detailed Calculations

Not applicable – this project is a software-only Java console application; no engineering calculations beyond programme logic (validation rules, category counts, and statistics computation) were required.

Appendix B – Engineering Drawings / Schematics

See Figure 3.1 (System Architecture Diagram) and Figures 4.1–4.4 (Module Flow Diagrams) in Chapters 3 and 4.

Appendix C – Source Code / Code Repository Information

Only essential design details are included in this report (Sections 3.5 and 4.2). The complete Java source code for all four modules (User & Waste Registration, Waste Collection Management, Waste Classification & Status, and Reports & Monitoring) is maintained in the team's approved project repository.

Appendix D – Datasheets

Not applicable – no external hardware components were used in this software-only prototype.

Appendix E – Experimental / Raw Data

Sample test cases and their pass/fail outcomes are summarised in Section 4.3 (Test Plan & Verification).

Appendix F – Ethics / Institutional Approval

Not applicable – the project does not involve human subjects, personal data collection beyond in-memory waste-request details, or animal studies.

Appendix G – Project Meeting / Progress Records

Milestone-level progress is summarised in the project planning table in Section 4.8.

Appendix H – User / Industry Feedback

Not applicable at this prototype stage.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable at this prototype stage.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (front matter) and the roles/responsibilities table in Section 4.8.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal / environmental impact	SO2 / SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2 / SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1 / SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)